

Rent-A-Camper

Konzeption und Entwicklung eines Full-Stack-Portals
zum Mieten von Wohnmobilen

Technische Dokumentation zum Projekt „wohnmobilapp“
Modul: Webbasierte Anwendungsentwicklung — 4. Semester
Repository: rentacamperapp (Full-Stack-TypeScript-Monorepo)

Autoren

Dominik Kollenz

Matrikelnr: 5288656

Kevin Gojani

Matrikelnr: 6602955

Datum

5. Juli 2026

Frontend: Vanilla TypeScript · eigene JSX-Runtime · Bootstrap 5 · Vite
Backend: Node.js · NestJS · Supabase/PostgreSQL
Deployment: Docker

Inhaltsverzeichnis

1	Einleitung	1
1.1	Projektüberblick	1
1.2	Umgesetzte Funktionalität	1
1.3	Zielsetzung dieses Dokuments	1
2	Laufzeitumgebung und Technologie-Stack	2
2.1	Backend: Node.js und NestJS	2
2.2	Frontend: Vanilla TypeScript mit eigener JSX-Runtime	3
2.3	Datenbank und externe Dienste	4
2.4	Deployment und Containerisierung	4
3	Monorepo- und Ordnerstruktur	4
3.1	Gesamtstruktur	4
3.2	Backend-Struktur	5
3.3	Frontend-Struktur	6
4	Architektur und angewandte Techniken	7
4.1	Backend-Architektur: Vom Request zur Datenbank	7
4.2	Das Repository-Pattern und Dependency-Injection-Tokens	7
4.3	Frontend-Architektur: Die eigene JSX-Runtime	8
4.4	Komponenten, Views und Services im Frontend	9
4.5	Zusammenspiel Frontend und Backend	9
5	Zeiterfassung und Projektverlauf	10
5.1	Stundenübersicht	10
5.2	Der Irrweg: Imperative DOM-Manipulation hinter einer JSX-Fassade	11
5.3	Die Lösung: Das große Architektur-Refactoring	13
6	Code-Analyse: Ausgewählte Implementierungen	14
6.1	Backend: Controller, Service und DTO im Buchungsmodul	14
6.2	Frontend: Eine rein deklarative JSX-Komponente	16
6.3	Dokumentationsstandard: JSDoc am Beispiel der JSX-Runtime	17
7	Bildschirmabgriffe	18
8	Fazit und Ausblick	22
8.1	Fazit	22
8.2	Ausblick	22

1 Einleitung

1.1 Projektüberblick

„Rent-A-Camper“ ist im Rahmen eines universitären Softwareprojekts im 4. Semester als webbasiertes Portal zum Mieten von Wohnmobilen entstanden. Nutzer können sich registrieren, verfügbare Fahrzeuge durchsuchen und filtern, einen Buchungszeitraum samt Zusatzleistungen (Add-ons) auswählen, den Mietpreis in Echtzeit berechnen lassen und die Buchung über PayPal oder Stripe bezahlen. Fahrzeughalter (Vermieter) können über einen eigenen Bereich ihre Wohnmobile anlegen, bearbeiten, mit Bildern versehen und Verfügbarkeiten blockieren.

Das Projekt ist bewusst als **Full-Stack-TypeScript-Monorepo** aufgesetzt: ein gemeinsamer Ordner `wohnmobilapp/` enthält sowohl das `backend/` (NestJS) als auch das `frontend/` (Vanilla TypeScript) sowie einen `shared/`-Ordner für Typen, die von beiden Seiten verwendet werden. Diese Struktur wurde gewählt, damit Änderungen an der Datenbank-Struktur (z. B. an generierten Supabase-Typen) an einer zentralen Stelle gepflegt und von Backend und Frontend gleichermaßen konsumiert werden können, ohne sie doppelt pflegen zu müssen.

Persistiert werden die Daten in einer PostgreSQL-Datenbank, gehostet über **Supabase**, das zugleich Authentifizierung (Bearer-Token-basiert) und Objekt-Storage für Bilder bereitstellt. Es steht eine vollständige Docker-Compose-Konfiguration für lokale bzw. selbstgehostete Deployments zur Verfügung.

1.2 Umgesetzte Funktionalität

Konkret wurde folgender Funktionsumfang implementiert und ist in der laufenden Anwendung nutzbar:

- Registrierung und Login über Supabase Auth (Bearer-Token-basiert).
- Suche und Filterung verfügbarer Wohnmobile nach Preis, Betten, Maßen, Kraftstoffart, Führerscheinklasse, Ausstattungsmerkmalen und Anbietertyp, inklusive Sortierung und Verfügbarkeitsprüfung über einen Datumsbereich.
- Detailansicht eines Wohnmobils mit Bildergalerie und Live-Preisberechnung (Saison-Zuschläge, Rabatte, Add-ons, Reinigungsgebühr, Kautions).
- Buchungsabschluss über einen mehrstufigen Checkout mit Auswahl von Abhol-/Rückgabeort, Zusatzleistungen und Zahlungsmethode (PayPal oder Stripe), inklusive automatischer PDF-Rechnungserstellung und E-Mail-Bestätigung.
- Konto-Bereich für Mieter:innen mit Übersicht und Stornierung eigener Buchungen.
- Vermieter-Bereich zum Anlegen, Bearbeiten und Löschen eigener Wohnmobile samt Bild-Upload, Ausstattungspflege und manueller Blockierung von Verfügbarkeitszeiträumen (z. B. für Wartung).
- Kontakt-/Support-Formular mit serverseitigem E-Mail-Versand.

1.3 Zielsetzung dieses Dokuments

Diese Dokumentation verfolgt drei Ziele. Erstens soll sie die technische Architektur des Systems — Laufzeitumgebung, Ordnerstruktur, Zusammenspiel von Backend und Frontend — so nachvollziehbar beschreiben, dass sich auch projektfremde Entwickler:innen zügig orientieren können. Zweitens dokumentiert sie anhand konkreter Code-Beispiele aus dem tatsächlichen Repository, welche Muster (Schichtenarchitektur,

Repository-Pattern, deklaratives Rendering) im Projekt etabliert wurden und warum. Drittens — und das ist uns besonders wichtig — reflektiert sie **ehrlich** den Entwicklungsprozess: welche Entscheidungen sich im Nachhinein als Sackgasse erwiesen haben, wie viel Zeit das gekostet hat und welche konkrete Lösung erarbeitet wurde. Kapitel 5 ist diesem Aspekt gewidmet und enthält auch die vollständige Zeiterfassung des Projekts.

2 Laufzeitumgebung und Technologie-Stack

Das Projekt trennt Backend und Frontend vollständig in zwei unabhängige Node.js-Prozesse, die ausschließlich über eine REST-Schnittstelle kommunizieren. Beide Teile sind in TypeScript geschrieben, verwenden jedoch unterschiedliche Frameworks bzw. — im Fall des Frontends — bewusst **kein** Framework für das UI-Rendering.

2.1 Backend: Node.js und NestJS

Das Backend läuft auf **Node.js 22** und ist mit **NestJS 11** umgesetzt, einem opinionated Framework, das Dependency Injection, Decorator-basiertes Routing und eine modulare Architektur nach dem Vorbild von Angular auf die Server-Seite bringt. Zentrale Pakete:

Paket	Verwendungszweck
@nestjs/core, @nestjs/common	Kern-Framework: Module, Controller, Dependency Injection, Guards
@nestjs/platform-express	HTTP-Adapter — NestJS läuft hier auf Express als darunterliegendem Server
@nestjs/config	Laden und global bereitstellen der .env-Konfiguration
@supabase/supabase-js	Datenbank-Client (PostgreSQL über Supabase), Auth-Verifikation
stripe, @paypal/checkout-server-sdk	Serverseitige Zahlungsabwicklung
pdfkit	Serverseitige Erzeugung von Rechnungs-PDFs für bestätigte Buchungen
resend	Transaktionaler E-Mail-Versand (Buchungsbestätigung, Stornierung)
axios, @nestjs/axios	Ausgehende HTTP-Aufrufe an Drittanbieter-APIs
reflect-metadata, rxjs	Laufzeit-Abhängigkeiten von NestJS selbst (Decorator-Metadaten, Observables)

Der Einstiegspunkt `backend/src/main.ts` lädt die Umgebungsvariablen, instanziiert die Anwendung aus dem Wurzel-Modul `AppModule`, aktiviert CORS (da Frontend und Backend auf unterschiedlichen Origins laufen) und startet den HTTP-Server auf dem in `PORT` konfigurierten Port (Standard: `3000`). Entwicklung erfolgt über `nest start --watch` mit automatischem Neustart bei Dateiänderungen; der Produktionsbuild wird mit `nest build` erzeugt und über `node dist/backend/src/main` gestartet.

2.2 Frontend: Vanilla TypeScript mit eigener JSX-

Runtime

Das Frontend verzichtet bewusst auf React, Vue oder ein vergleichbares Framework. Stattdessen kompiliert **Vite 8** TypeScript-Dateien mit JSX-Syntax (`.tsx`), wobei der TypeScript-Compiler so konfiguriert ist, dass er JSX-Ausdrücke nicht — wie sonst üblich — in Aufrufe von `React.createElement` übersetzt, sondern in Aufrufe einer **selbstgeschriebenen** Funktion. Das wird in `frontend/tsconfig.json` über zwei Compiler-Optionen festgelegt:

```
{
  "compilerOptions": {
    "jsx": "react",
    "jsxFactory": "createElement",
    "jsxFragmentFactory": "Fragment"
  }
}
```

Jede `<div>...</div>`-Notation im Code wird dadurch zu einem Aufruf von `createElement("div", attrs, ...children)`, implementiert in `frontend/src/utils/createElement.ts` (siehe Kapitel 4.3 und 6.3 für Details und den vollständigen Quellcode). Es gibt somit **kein virtuelles DOM** und **kein Diffing** — jeder Aufruf erzeugt sofort echte DOM-Knoten. Das ist die zentrale technische Eigenheit dieses Projekts und prägt seine gesamte Frontend-Architektur.

Weitere zentrale Frontend-Abhängigkeiten:

Paket	Verwendungszweck
vite	Dev-Server, Bundler; als Multi-Page-Application konfiguriert (siehe unten)
bootstrap	Utility-First-CSS-Klassen für Layout, Grid, Komponenten (Cards, Modals, Buttons)
sass	Kompiliert das eigene SCSS-Theme (<code>src/scss/</code>), das Bootstrap-Variablen überschreibt
flatpickr	Kalender-Widget für die Datumsauswahl bei Buchungen
@popperjs/core	Positionierungs-Engine für Dropdowns/Tooltips (Bootstrap-Abhängigkeit)
@paypal/paypal-js	Client-seitiges Laden des PayPal-Buttons im Checkout

Vite ist in `frontend/vite.config.js` explizit als **Multi-Page-Application** konfiguriert: statt einer einzigen `index.html` mit clientseitigem Routing (wie z. B. bei einer typischen React-SPA) besitzt jede View ihre eigene `index.html` und ihren eigenen TypeScript-Einstiegspunkt. Der Build-Prozess erzeugt daraus separate Bundles für `main`, `rent`, `rentout`, `aboutus`, `account`, `camperDetails` und `contact`. Ein zusätzliches Vite-Plugin (`camperDetailsRewritePlugin`) sorgt dafür, dass URLs der Form `/campers/<id>` im Dev-

Server auf die Detailseite umgeschrieben werden, damit die Camper-ID als Teil des Pfads (statt als Query-Parameter) funktioniert.

2.3 Datenbank und externe Dienste

Als Datenbank kommt **PostgreSQL** zum Einsatz, betrieben als Managed-Service über **Supabase**. Supabase übernimmt im Projekt drei Rollen gleichzeitig:

1. **Datenhaltung** — sämtliche fachlichen Tabellen (Camper, Buchungen, Add-ons, Preisregeln, Standorte, Profile, Führerscheinklassen, Blockierungen) liegen in Supabase/Postgres und werden über den offiziellen JS-Client (@supabase/supabase-js) sowohl aus dem Backend als auch — für öffentliche, lesende Zugriffe — teilweise direkt angesprochen.
2. **Authentifizierung** — Supabase Auth stellt Bearer-Tokens aus, die der AuthGuard des Backends (backend/src/modules/auth/auth.guard.ts) bei geschützten Routen verifiziert (siehe Kapitel 4.1).
3. **Typgenerierung** — über das Skript backend/scripts/sync-types.js (Aufruf: npm run sync-types) werden die aktuellen Datenbank-Typen aus Supabase gezogen, nach backend/src/types/supabase.ts geschrieben und von dort nach frontend/src/types/supabase.ts kopiert, sodass Backend und Frontend konsistent typisiert bleiben (siehe auch der ehrliche Hinweis zum shared/-Ordner in Kapitel 3.1).

Für Zahlungen werden **zwei** Anbieter parallel unterstützt — **Stripe** und **PayPal** — jeweils über eigene Service-Klassen im Backend (stripe.service.ts, paypal.service.ts), die hinter einem gemeinsamen PaymentsController liegen. Rechnungen werden serverseitig mit pdfkit als PDF erzeugt und Bestätigungs-/Stornierungs-E-Mails über **Resend** verschickt.

2.4 Deployment und Containerisierung

Für produktionsnahe, reproduzierbare Deployments existieren zwei mehrstufige Dockerfiles (Dockerfile.backend, Dockerfile.frontend) sowie eine docker-compose.yml im Repository-Wurzelverzeichnis. Eine Besonderheit: **der Build-Kontext beider Images ist zwingend das Repository-Wurzelverzeichnis**, nicht die jeweiligen Unterordner — weil der Backend-Build auch den shared/-Ordner mitkompiliert (dieser liegt relativ zu backend/ über ../shared im TypeScript-Include-Pfad) und daher im selben Build-Kontext liegen muss. Das Frontend-Image erhält seine Backend-URL (VITE_BACKEND_URL) als Docker-Build-Argument, da Vite VITE_*-Umgebungsvariablen zur Build-Zeit in das statische Bundle einbrennt — eine zur Laufzeit gesetzte Umgebungsvariable hätte hier keine Wirkung.

3 Monorepo- und Ordnerstruktur

3.1 Gesamtstruktur

Das Repository gliedert sich in drei unabhängig versionierte, aber im selben Monorepo verwaltete Bereiche:

```

wohnmobilapp/
├── backend/      NestJS-Anwendung (REST-API)
├── frontend/     Vanilla-TS-Anwendung (Vite, eigene JSX-Runtime)
└── shared/       Ursprünglich für geteilte Typen gedacht (siehe Hinweis)
```

```

unten)
├─ docker-compose.yml  Lokale/produktive Full-Stack-Orchestrierung
├─ Dockerfile.backend
├─ Dockerfile.frontend
├─ docs/               Diese Dokumentation (Typst-Projekt)

```

Ehrlicher Hinweis zum `shared/-Ordner

Der `shared/-` Ordner ist als „von Backend und Frontend gemeinsam genutzte Typen“ gedacht — in der Praxis ist er mittlerweile weitgehend **totes Gewicht**. Beide Seiten importieren ihre Supabase-Typen über eine eigene, lokale Kopie (`backend/src/types/supabase.ts` bzw. `frontend/src/types/supabase.ts`); der `@shared/*`-Pfad-Alias ist zwar in beiden `tsconfig.json`-Dateien sowie in `vite.config.js` konfiguriert, wird aber in keiner einzigen Quelldatei tatsächlich importiert. Die Datei `shared/types/supabase.ts` selbst ist zudem veraltet (295 statt aktuell 676 Zeilen) und wird vom Sync-Skript nicht mehr aktualisiert. Historisch enthielt der Ordner mehr (`camper.ts`, `insult.ts`), die im großen Refactoring (Kapitel 5.3) ersatzlos gestrichen wurden. Trotzdem ist der Ordner nicht gefahrlos entfernbar: `backend/tsconfig.json` inkludiert `../shared/**/*` im Kompilat, und `Dockerfile.backend` kopiert ihn explizit in den Build-Kontext — ein Löschen ohne begleitende Anpassung dieser beiden Stellen würde den Build brechen. Der Ordner ist damit ein weiteres, bewusst benanntes Beispiel für unvollständig nachgezogene Aufräumarbeit (vgl. die `api/-` Doppelstruktur in Kapitel 3.3).

3.2 Backend-Struktur

Das Backend folgt innerhalb von `backend/src/` einer klaren Trennung nach **fachlicher Domäne** (`modules/`) und **technischer Schicht** (`domain/`, `infrastructure/`):

```

backend/src/
├─ main.ts                Bootstrap: NestFactory, CORS, Port
├─ app.module.ts          Wurzel-Modul, importiert alle Feature-Module
├─ domain/               Reine Geschäftslogik, unabhängig vom Framework
│   ├── interfaces/      Fachliche Typ-Verträge (Booking, Location, ...)
│   └── services/        z.B. CamperPricingCalculator (reine Preis-
Mathematik)
│   └─ exceptions/       Fachliche Exceptions
├─ infrastructure/       Datenzugriff über Supabase, hinter Interfaces
│   └─ repositories/     gekapselt
├─ supabase/             SupabaseService (Client-Wrapper) + SupabaseModule
├─ types/                generierte Supabase-Datenbanktypen (sync-types.js)
└─ modules/              Ein Ordner pro fachlicher Domäne (Feature-Module)
    ├── auth/             AuthController, AuthGuard, AuthService
    ├── campers/           Controller + Service (camper.module.ts)
    ├── camper_images/
    ├── camper_features/
    └── camper_blockings/  Manuelle Sperrzeiträume eines Fahrzeugs

```

└─ camper_owner/	
└─ addons/	Katalog buchbarer Zusatzleistungen
└─ booking_addons/	n:m-Verknüpfung Buchung <-> Add-on
└─ bookings/	+ dto/create-booking.dto.ts
└─ pricing_rules/	
└─ drivers_license/	Führerscheinklassen-Validierung
└─ profiles/	
└─ payments/	stripe.service.ts, paypal.service.ts
└─ locations/	Abhol-/Rückgabeorte
└─ mail/	Resend-Anbindung
└─ pdf/	pdfkit-Rechnungserzeugung
└─ support/	+ dto/contact-request.dto.ts

Jedes Feature-Modul folgt demselben, in NestJS üblichen Dreiklang: ein `*.module.ts` (Verdrahtung/Dependency Injection), ein `*.controller.ts` (HTTP-Schicht) und ein `*.service.ts` (Geschäftslogik). Module mit komplexeren Eingabedaten besitzen zusätzlich einen `dto/-`Unterordner. Details zum Zusammenspiel dieser Bausteine folgen in Kapitel 4.1.

3.3 Frontend-Struktur

Das Frontend trennt **Einstiegspunkte** (`pages/`), **Komponenten** (`components/`) und die neuere, schichtenorientierte Struktur (`domain/`, `infrastructure/`), die im Zuge des in Kapitel 5 beschriebenen Refactorings eingeführt wurde:

frontend/src/	
└─ index.html, home.ts	Einstiegspunkt: Startseite
└─ jsx.d.ts	JSX-Typdeklarationen für den TS-Compiler
└─ pages/	Ein Ordner pro Route (Vite Multi-Page-App)
└─┬─ rent/	index.html + rent.ts -> komponiert die View
└─┬─ rentout/	
└─┬─ camper-details/	
└─┬─ checkout/, checkout-success/	
└─┬─ account/, aboutus/, contact/	
└─ components/	Die eigentlichen JSX-Views/-Komponenten
└─┬─ rent/	RentPage, CamperCard, FilterBar, filter-bar/*
└─┬─ rentout/	RentoutPage, CamperCRUD, camper-crud/*
└─┬─ camper-details/	BookingCard, ImageGallery, booking-card/*
└─┬─ checkout/	CheckoutPage, checkout-page/*
└─┬─ account/	AccountPage, BookingsTable, bookings-table/*
└─┬─ auth/, common/, home/, aboutus/	
└─ domain/	Modelle, Validatoren, reine Fach-Services
└─┬─ models/	z.B. auth.model.ts
└─┬─ services/	z.B. CamperFilterService (reine Filter-/
Sortierlogik)	
└─┬─ validators/	credential-validator.ts, drivers-license-
validator.ts	
└─ infrastructure/	Technische Anbindung nach außen
└─┬─ api/	Ein API-Client pro Backend-Ressource
(Klassen)	
└─┬─ transformers/	DTO -> Domänenmodell-Mapper
└─┬─ session-storage.ts	
└─ api/	Ältere, funktionale API-Wrapper


```
(campersAPI.ts, ...)
├─ ui/helpers/           UI-Hilfsfunktionen
├─ utils/               createElement.ts (JSX-Runtime), mockData.ts
├─ types/, scss/, fonts/
```

Ein ehrlicher Hinweis vorweg: `api/` (funktionale Wrapper wie `campersAPI.ts`) und `infrastructure/api/` (neuere, klassenbasierte API-Clients wie `camper-api-client.ts`) existieren aktuell **nebeneinander**. Das ist keine geplante Doppelstruktur, sondern das sichtbare Ergebnis eines Refactorings, das zum Abgabzeitpunkt begonnen, aber noch nicht flächendeckend abgeschlossen wurde — dazu mehr in Kapitel 5.3.

4 Architektur und angewandte Techniken

4.1 Backend-Architektur: Vom Request zur Datenbank

Das Backend ist als klassische **Schichtenarchitektur** mit vier Ebenen aufgebaut, die in dieser Reihenfolge durchlaufen werden:

1. **Controller** — nimmt den HTTP-Request entgegen, extrahiert Pfad-/Body-Parameter über NestJS-Decorators (`@Param()`, `@Body()`) und validiert höchstens die **Form** der Anfrage. Enthält bewusst keine Geschäftslogik.
2. **Service** — orchestriert die eigentliche Anwendungslogik: ruft ein oder mehrere Repositories auf, wendet fachliche Regeln an und wirft domänenspezifische Exceptions.
3. **Domain** — enthält reine, framework-unabhängige Berechnungen und Regeln (z. B. die Preisberechnung), die ohne NestJS und ohne Datenbankzugriff testbar sind.
4. **Infrastructure/Repository** — kapselt den tatsächlichen Zugriff auf Supabase/PostgreSQL hinter einem Interface, sodass Services nicht direkt von der konkreten Datenbank-Implementierung abhängen.

Diese Trennung zeigt sich exemplarisch am Buchungsmodul. Der `BookingsController` (siehe Codebeispiel in Kapitel 6.1) delegiert jede Anfrage unmittelbar an den `BookingsService`; dieser wiederum bezieht seine Abhängigkeiten — Repository, Führerschein-Prüfung, Profil-Service, PDF-/Mail-Versand — nicht direkt, sondern über das zugehörige `BookingsModule`, das sie per Dependency Injection bereitstellt.

4.2 Das Repository-Pattern und Dependency-Injection-Tokens

Ein wiederkehrendes Muster im Backend ist die Entkopplung von Services und konkreten Datenzugriffsklassen über **Interfaces plus DI-Tokens**. Statt einen Service direkt von `BookingRepository` (der Supabase-Implementierung) abhängen zu lassen, hängt er von einem Interface `IBookingRepository` ab. Das zugehörige Modul bindet dieses Interface zur Laufzeit an die konkrete Klasse:

```
providers: [  
  BookingsService,  
  {  
    provide: BOOKING_REPOSITORY_TOKEN,  
    useClass: BookingRepository,  
  },  
  BookingRepository,  
],
```

Der Vorteil: Services (CamperService, BookingsService, ...) sind gegenüber der konkreten Datenbanktechnologie austauschbar — ein Test könnte den Token stattdessen an ein In-Memory-Repository binden, ohne den Service selbst anzufassen. Dieses Muster wird konsequent auch für die Preisregeln- und Add-on-Repositories angewendet (PRICING_RULE_REPOSITORY_TOKEN, ADDON_REPOSITORY_TOKEN), wie in `camper.service.ts` zu sehen ist.

Ergänzend gibt es eine dedizierte **Domain-Schicht** für Logik, die absichtlich **keinen** Bezug zu NestJS oder Supabase hat: `CamperPricingCalculator` erhält Basispreis, Reinigungsgebühr, Kautions, Zeitraum, Preisregeln und Add-ons als reine Werte und liefert eine vollständige Preisaufschlüsselung zurück — ohne selbst irgendetwas zu laden. `CamperService.calculatePrice()` (siehe Kapitel 6, `camper.service.ts`) beschafft lediglich die benötigten Daten aus den Repositories und reicht sie an den Calculator weiter. Das macht die eigentliche Preislogik isoliert unit-testbar.

4.3 Frontend-Architektur: Die eigene JSX-Runtime

Das technisch prägendste Merkmal des Frontends ist der Verzicht auf ein UI-Framework zugunsten einer selbstgeschriebenen, rund 170 Zeilen kurzen JSX-Runtime (`frontend/src/utils/createElement.ts`, vollständig abgedruckt in Kapitel 6.3). Sie übernimmt exakt die Aufgabe, die sonst React mit seinem virtuellen DOM übernimmt — mit einem entscheidenden Unterschied: es gibt kein virtuelles DOM und kein Diffing.

`createElement(tag, attrs, ...children)` erzeugt bei jedem Aufruf sofort einen echten `document.createElement(...)`-Knoten, setzt die übergebenen Attribute direkt darauf (inklusive React-ähnlicher Komfortfunktionen wie `className`, `style={{...}}` als Objekt und `onClick={...}` als Event-Listener) und hängt die Kinder synchron an.

Diese Entscheidung hat unmittelbare Konsequenzen für den Programmierstil:

- **Komponenten sind Funktionen, die einmalig ein DOM-Element zurückgeben.** Es gibt keinen Re-Render-Zyklus — wenn sich Daten ändern, muss der betroffene Teilbaum **explizit** neu erzeugt und ausgetauscht werden (siehe die Diskussion des „Irrwegs“ in Kapitel 5.2, in dem genau das anfangs fehlerhaft gelöst wurde).
- **Bedingtes Rendering** funktioniert über dieselben JavaScript-Ausdrücke wie in React (`{condition && <Element/>}`), weil `appendChild()` in der Runtime `null/undefined/false`-Kinder explizit überspringt.
- **Fragmente** (`<>...</>`) werden über ein `Symbol` als Marker realisiert und liefern ein natives `DocumentFragment` zurück — nützlich für Komponenten wie `PriceBreakdownList`, die mehrere Geschwister-Elemente ohne umschließendes Wrapper-Element rendern müssen (siehe Kapitel 6.2).

4.4 Komponenten, Views und Services im Frontend

Der Aufbau einer typischen Seite folgt einem festen Muster: eine `pages/<name>/index.html` bindet ein schlankes Einstiegs-Skript ein (z. B. `pages/rent/rent.ts`), das lediglich `MainHeader()`, die eigentliche Seiten-Komponente (z. B. `RentPage()`) und `MainFooter()` erzeugt und an `document.body` anhängt:

```
document.body.appendChild(MainHeader());
document.body.appendChild(RentPage());
document.body.appendChild(MainFooter());
```

Die eigentliche Seiten-Komponente (`components/rent/RentPage.tsx`) baut die JSX-Struktur auf, zerlegt größere Bereiche in kleinere Unterkomponenten (`components/rent/filter-bar/*`, `CamperCard.tsx`) und lädt die Daten asynchron über die **API-Client-Schicht** nach. Diese Schicht existiert in zwei Generationen: die älteren, funktionalen Wrapper unter `src/api/*API.ts` (z. B. `getAllCampers()` in `campersAPI.ts`) und die neueren, klassenbasierten Clients unter `src/infrastructure/api/*-client.ts`, die auf einem gemeinsamen `BaseApiClient` aufbauen und konsistent Fehlerbehandlung sowie Auth-Header setzen. Fachliche Berechnungen, die nichts mit dem DOM zu tun haben — etwa das Filtern und Sortieren der Camper-Liste — sind bewusst in die `domain/services/-`Schicht ausgelagert (`CamperFilterService`), damit sie unabhängig von der konkreten View getestet werden könnten.

4.5 Zusammenspiel Frontend und Backend

Frontend und Backend kommunizieren ausschließlich über eine REST-Schnittstelle über HTTP/JSON; es gibt keine gemeinsame Prozessgrenze und keinen Server-Side-Rendering-Schritt. Der Ablauf einer typischen Aktion — etwa das Laden der Camper-Liste auf der Miet-Seite — sieht wie folgt aus:

1. Die Komponente `RentPage.tsx` ruft nach dem ersten Rendern (verzögert über `setTimeout(loadCampers, 0)`, damit das Grundgerüst inklusive Skeleton-Karten sofort sichtbar ist) `getAllCampers()` auf.
2. Der API-Client sendet einen GET-Request an die entsprechende Route des NestJS-Backends (`CampersController`).
3. Der Controller delegiert an `CampersService.findAllCampers()`, der über `CampersRepository` auf Supabase zugreift.
4. Die JSON-Antwort durchläuft im Frontend ggf. einen **Transformer** (`infrastructure/transformers/`), der das Rohformat der API in ein Domänenmodell überführt, bevor die Liste zur Anzeige an `CamperCard()` übergeben wird.

Für zustandsbehaftete Vorgänge wie den Checkout wird zusätzlich `sessionStorage` (`infrastructure/session-storage.ts`) genutzt, um Buchungsdetails über einen Redirect zum externen Zahlungsanbieter (PayPal) hinweg zwischenspeichern — da beim Zurückkehren von PayPal die JavaScript-Laufzeit der Seite komplett neu initialisiert wird.

5 Zeiterfassung und Projektverlauf

5.1 Stundenübersicht

Die folgende Tabelle fasst die investierte Arbeitszeit über den gesamten Projektzeitraum (18. Mai bis 5. Juli 2026) zusammen, wie sie sich aus Commit-Historie, Pull-Requests und ergänzender manueller Aufzeichnung rekonstruieren lässt. Sie ist bewusst granular gehalten, weil gerade die Verteilung zwischen den Kategorien — insbesondere der hohe Anteil an Refactoring und Bughunting in der zweiten Projekthälfte — der eigentliche Beleg für den in Kapitel 5.2 beschriebenen Irrweg ist.

Datum	Person	Tätigkeit	Kategorie	Std.
18.05.	Dominik	Projekt-Kickoff: Repository-Setup, Vite-Scaffold, erste <code>index.html</code> ; README mit Tech-Stack und Deploy-Links	Onboarding/Setup	5.5
19.05.	Dominik	Grundstruktur für Backend und Frontend angelegt (5 Zwischen-Commits)	Onboarding/Setup	6.0
20.05.	Kevin	Header-Icon und erste statische UI-Bausteine	Frontend-Views	2.0
21.05.	Dominik	Migration des Backends von Express auf NestJS (Modul-/Controller-Grundgerüst)	Backend-Architektur	6.5
21.05.	Dominik	Dockerfiles/ <code>docker-compose.yml</code> ; README-Update (Vercel/Heroku/Supabase-Links)	Deployment	4.0
31.05.	Dominik	Bootstrap-5-Integration, SCSS-Theme-Variablen	Frontend-Views	4.5
09.06.	Dominik	Homepage-Design: Hero, Highlights, Workflow-Sektion, Suchleiste	Frontend-Views	7.0
14.06.	Kevin	Anpassungen an der Startseite (Issue #16)	Frontend-Views	3.0
16.06.	Dominik	Umstieg der Startseite von HTML-Strings auf TSX; eigene JSX-Runtime <code>createElement.ts</code> grundlegend erstellt	Frontend-Architektur	8.0
17.06.	Dominik	Ordnerstruktur überarbeitet (Trennung <code>components/pages</code> , erste Domain-Ordner)	Refactoring	4.0
23.06.	Dominik	Erste Frontend-Backend-Kommunikation (<code>fetch-Wrapper</code> , <code>campersAPI.ts</code>)	Backend-Architektur	6.5
25.06.	Dominik	Camper-Detailseite inkl. Bildergalerie und Buchungswidget	Frontend-Views	6.0
30.06.	Kevin	„Refactoring“ der Rent-Page — tatsächlich Einführung von <code>container</code> -Variablen, verschachtelten <code>querySelector</code> -Aufrufen und manuellem <code>innerHTML</code> -Handling (siehe 5.2)	Frontend-Views	7.5

01.07.	Kevin	Umstellung von Mock-Daten auf echte Datenbankdaten innerhalb der bereits unübersichtlichen Struktur	Frontend-Views	5.0
01.07.	Dominik	Bughunting: fehlerhafte DOM-Referenzen nach dynamischem Neu-Rendern des Grids, Race-Conditions bei <code>setTimeout(loadCampers, 0)</code>	Bughunting	3.5
02.07.	Dominik	Großes Architektur-Refactoring: Domain-/Infrastructure-Schicht, API-Client-Klassen, Validatoren, Rückbau der <code>container</code> -Anti-Pattern (siehe 5.3)	Refactoring	9.5
02.07.	Dominik	Zahlungsanbindung: PayPal- und Stripe-Integration	Backend-Architektur	6.0
03.07.	Dominik	Bugfixes, fehlende Pflichtfeatures nachgezogen, UI-Feinschliff	Bughunting	7.0
04.07.	Dominik	Feinschliff: responsive Anpassungen, Randfälle bei der Preisberechnung, Cross-Browser-Test	Refactoring	6.5
04.07.	Dominik	Finale Politur vor Abgabe (Deadline-Endspurt, Commit „holy moly“)	Bughunting	4.5
05.07.	Dominik, Kevin	Verfassen der technischen Dokumentation (dieses Dokument)	Doku	5.0

Gesamtaufwand: 117,5 Personenstunden

davon Dominik: 100,0 Std. · Kevin: 17,5 Std.

5.2 Der Irrweg: Imperative DOM-Manipulation hinter einer JSX-Fassade

Der ehrlichste Teil dieser Dokumentation betrifft eine Designentscheidung, die sich über gut zwei Wochen (ca. 25.06. bis 01.07.) unbemerkt in den Code einschlich und deren Aufräumen anschließend den mit Abstand größten Einzelposten der Zeiterfassung verursachte: den Commit `Refactored Pages and Build Rent` vom 30.06.2026.

Auf den ersten Blick sah der Code sauber aus — er verwendete durchgehend JSX-Syntax. Das eigentliche Problem lag darin, **wie** diese JSX-Bäume anschließend behandelt wurden. Statt eine Komponente rein deklarativ aus den Daten abzuleiten, wurde das erzeugte Wurzelelement in einer Variable namens `container` zwischengespeichert und danach **imperativ** wiederholt durchsucht und mutiert:

```
export function RentPage() {
  let campersList: any[] = [];

  const container = (
    <div className="container-fluid py-5 px-3 px-md-5">
```

```

    { /* ... */ }
    <div className="row" id="camper-grid-container"></div>
  </div>
) as HTMLElement;

const updateCampersList = () => {
  const form = container.querySelector("form") as HTMLFormElement;
  // ... rund 15 weitere querySelector/namedItem-Zugriffe auf
  Formularfelder ...

  let filtered = campersList.filter((camper) => { /* ... 40 Zeilen
  Filterlogik ... */ });

  const grid = container.querySelector("#camper-grid-container") as
  HTMLElement;
  const emptyState = container.querySelector("#empty-state") as HTMLElement;
  const resultsCount = container.querySelector("#results-count") as
  HTMLElement;

  grid.innerHTML = "";
  resultsCount.textContent = `${filtered.length} Fahrzeuge gefunden`;
  filtered.forEach((camper) => grid.appendChild(CamperCard(camper)));
};

const filterSidebar = container.querySelector("#filter-sidebar") as
HTMLElement;
filterSidebar.appendChild(FilterBar({ onFilterChange: updateCampersList }));

setTimeout(loadCampers, 0);
return container;
}

```

⚠️ Warum das ein Irrweg war

Diese Struktur **sieht** aus wie eine deklarative Komponente, **verhält** sich aber wie klassisches jQuery-Styling aus den 2010er-Jahren — mit JSX nur als Fassade für den initialen Aufbau:

- Die 40-zeilige Filterlogik, die eigentlich reine, testbare Geschäftslogik ist, war **inline** in einer Closure innerhalb der Komponente vergraben — sie ließ sich weder wiederverwenden noch isoliert testen.
- Jede Interaktion mit dem gerenderten Baum lief über `container.querySelector(...)` mit String-IDs (`"#camper-grid-container"`, `"#filter-sidebar"`, ...). Ein Tippfehler in einem String fällt nicht beim Kompilieren auf, sondern erst zur Laufzeit als `null`-Dereferenzierung.
- `grid.innerHTML = ""` gefolgt von manuellem erneuten `appendChild()` ist exakt die Art von DOM-Mutation, die eine JSX-Runtime eigentlich **vermeiden** soll — der ganze Sinn deklarativen Renderings geht verloren, sobald man das Ergebnis danach wieder wie klassisches `innerHTML` behandelt.

- `let campersList: any[] = []` — die zentrale Datenstruktur der Seite war zum Zeitpunkt dieses Commits vollständig ungetypt.
- Der `setTimeout(loadCampers, 0)`–„Hack“, um Daten nach dem ersten Rendern nachzuladen, führte in Kombination mit dem direkten DOM-Zugriff zu echten Bugs: Wenn Nutzer:innen den Filter änderten, während `loadCampers()` noch lief, griffen zwei nebenläufige Funktionen auf denselben `container` zu — genau die Race-Condition, die am 01.07. unter „Bug hunting“ protokolliert ist.

Der Kern des Problems: **containerbasierte, imperative DOM-Manipulation hinter einer JSX-Oberfläche vereint die Nachteile beider Welten** — man verliert die Übersichtlichkeit von reinem HTML/Vanilla-JS, ohne die Sicherheit eines echten deklarativen Modells (Typprüfung, Testbarkeit, Wiederverwendbarkeit) zu gewinnen.

5.3 Die Lösung: Das große Architektur-Refactoring

Am 02.07.2026 — zwei Tage vor der ursprünglich geplanten Feinschliff-Phase — wurde die Entscheidung getroffen, nicht länger einzelne Symptome zu flicken, sondern die zugrunde liegende Struktur in einem gebündelten Refactoring (Commit `Major Refactoring & Added Paypal/stripe integration`) zu bereinigen. Dieser eine Commit veränderte 129 Dateien (+4729/–1718 Zeilen) und legte den Grundstein für die in Kapitel 3.3 und 4.3–4.4 beschriebene Schichtenstruktur. Konkret wurden folgende Prinzipien durchgesetzt:

1. **Strikte TypeScript-Typisierung statt `any`.** Fachliche Datenstrukturen (`MockCamper`, `AuthModel`, Filterkriterien) erhielten explizite Interfaces; `noUnusedLocals`/`noUnusedParameters` im `tsconfig.json` wurden bereits vorher scharf gestellt, wodurch tote Variablen jetzt zuverlässig auffallen.
2. **Auslagerung von Geschäftslogik in eigene Dateien nach Java-artigem Objektprinzip** — eine Verantwortlichkeit, eine Klasse, eine Datei. Die 40-zeilige Inline-Filterfunktion aus dem Irrweg wurde zu `CamperFilterService.filterAndSort()` (`domain/services/camper-filter.service.ts`), einer zustandslosen, unabhängig aufrufbaren statischen Methode mit vollständig typisiertem `CamperFilterCriteria`-Parameter.
3. **Konsequentes File-Splitting großer Komponenten.** Statt einer monolithischen `RentPage.tsx`, die Formular, Grid und Sortierung in einer Datei vermischte, entstanden fokussierte Unterkomponenten (`filter-bar/FilterDropdown.tsx`, `filter-bar/RangeInputPair.tsx`, `filter-bar/FeatureCheckboxList.tsx`), die unabhängig lesbar und testbar sind. Dasselbe Muster wurde für die Buchungsseiten (`booking-card/*`, `checkout-page/*`, `bookings-table/*`, `camper-crud/*`) angewendet.
4. **Eine klassenbasierte API-Schicht** (`infrastructure/api/*-client.ts`) mit einem gemeinsamen `BaseApiClient`, der Fehlerbehandlung, Basis-URL und Auth-Header zentral kapselt, statt in jeder `*API.ts`-Datei erneut zu duplizieren.
5. **Möglichst rein deklaratives Rendering.** Wo immer sinnvoll möglich, wurde der `container`-Zugriff durch echte JSX-Kompositionsmuster ersetzt — am konsequentesten umgesetzt in kleineren, neu entstandenen Komponenten wie `PriceBreakdownList.tsx` (vollständiges Beispiel in Kapitel 6.2), die ausschließlich aus Props ein Ergebnis ableiten, ohne jemals selbst `querySelector` aufzurufen.

✓ Ergebnis und ehrliche Einordnung

Das Refactoring hat die schlimmsten Symptome beseitigt und der Codebasis eine klare Schichtung gegeben, die neue Features seither deutlich schneller umsetzbar macht. **Es ist jedoch kein vollständiger Neuanfang gewesen:** `RentPage.tsx` selbst enthält auch nach dem Refactoring noch `container.querySelector(...)`-Aufrufe und ein `grid.innerHTML = ""` (siehe Kapitel 3.3, Hinweis zur Doppelstruktur `api/` vs. `infrastructure/api/`). Die Filterlogik wurde ausgelagert und typisiert, die Rendering-Strategie der Ergebnisliste selbst jedoch aus Zeitgründen bis zur Abgabe nicht vollständig auf ein deklaratives Diffing-Muster umgestellt. Diese verbleibende Altlast ist bewusst **nicht** verschwiegen, sondern als offener Punkt im Ausblick (Kapitel 8) vermerkt — echtes Engineering-Wachstum bedeutet für uns auch, ehrlich zu benennen, was noch nicht fertig ist, statt technische Schulden zu kaschieren.

6 Code-Analyse: Ausgewählte Implementierungen

Dieses Kapitel zeigt drei kuratierte Ausschnitte aus dem tatsächlichen Quellcode, die aus unserer Sicht exemplarisch für den nach dem Refactoring etablierten Qualitätsstandard stehen.

6.1 Backend: Controller, Service und DTO im Buchungsmodul

Das folgende Beispiel zeigt den vollständigen `BookingsController`. Er ist bewusst „dünn“ gehalten: jede Methode validiert höchstens die Form der Anfrage (z. B. die Pflichtprüfung der Nutzer-ID bei der Stornierung) und delegiert die eigentliche Arbeit vollständig an `BookingsService`.

```
@Controller('bookings')
export class BookingsController {
  constructor(private readonly bookingsService: BookingsService) {}

  /**
   * `POST /bookings/create` — creates a new booking.
   *
   * @param dto The booking request body.
   * @returns The newly created booking.
   */
  @Post('create')
  async create(@Body() dto: CreateBookingDto): Promise<unknown> {
    return await this.bookingsService.createBooking(dto);
  }

  /**
   * `POST /bookings/:id/cancel` — cancels a booking on behalf of its owner.
   */
}
```



```

* @param id Id of the booking to cancel.
* @param dto Body carrying the requesting user's id (used for the ownership
*           check inside the service).
* @returns The cancelled booking.
* @throws ForbiddenException If no user id is supplied.
*/
@Post('/:id/cancel')
async cancelBooking(
  @Param('id') id: string,
  @Body() dto: CancelBookingDto,
): Promise<unknown> {
  if (!dto.user_id) {
    throw new ForbiddenException('User ID is required');
  }
  return await this.bookingsService.cancelBooking(id, dto.user_id);
}

```

Dazu gehört das passende Data-Transfer-Object, das die erwartete Form des Anfrage-Bodys deklariert:

```

/**
 * Request body for creating a booking (`POST /bookings/create`).
 *
 * Uses snake_case to match the JSON the frontend sends. The service maps this
 * into the internal `CreateBookingCommand` before persisting.
 *
 * NOTE: there are no `class-validator` decorators yet, so these fields are not
 * automatically validated at the HTTP boundary – the service performs the
 * business validation (dates, license, availability) explicitly.
 */
export class CreateBookingDto {
  camper_id: string;
  user_id: string;
  start_date: string;
  end_date: string;
  total_price: number;
  addons?: string[];
  pickup_location_id?: string;
  return_location_id?: string;
}

```

Ehrlicher Hinweis zur DTO-Validierung

Der Code-Kommentar im DTO ist absichtlich unverändert übernommen: Es werden aktuell **keine** class-validator-Decorators (@IsString(), @IsNotEmpty(), ...) eingesetzt, wie man sie in einem NestJS-Lehrbuch erwarten würde. Stattdessen übernimmt BookingsService die fachliche Validierung (Datumsformate, Führerscheinklasse, Verfügbarkeit) explizit im Code. Das ist eine bewusste, aber

dokumentierte Lücke: für ein produktives System wäre die deklarative Validierung direkt an der HTTP-Grenze über `class-validator` und einen globalen `ValidationPipe` nachzurüsten.

Wie diese Bausteine per Dependency Injection zusammengeführt werden, zeigt das zugehörige Modul (gekürzt):

```
@Module({
  imports: [
    SupabaseModule,
    DriversLicenseModule,
    ProfilesModule,
    CampersModule,
    CamperBlockingsModule,
    PdfModule,
    MailModule,
  ],
  controllers: [BookingsController],
  providers: [
    BookingsService,
    { provide: BOOKING_REPOSITORY_TOKEN, useClass: BookingRepository },
    BookingRepository,
  ],
  exports: [BookingsService],
})
export class BookingsModule {}
```

6.2 Frontend: Eine rein deklarative JSX-Komponente

Als Gegenstück zum in Kapitel 5.2 gezeigten Irrweg dient `PriceBreakdownList.tsx` als Beispiel für eine Komponente, die **ausschließlich** aus ihren Props ein Ergebnis ableitet — ohne eine einzige `querySelector`- oder `innerHTML`-Anweisung:

```
import { createElement, Fragment } from "../../utils/createElement.ts";

export interface PriceBreakdownRow {
  label: string;
  value: string;
  textClass?: string;
}

interface PriceBreakdownListProps {
  rows: PriceBreakdownRow[];
}

/**
 * Renders a list of price-breakdown rows (<li> elements only — no wrapping
 * <ul>), used by both the booking widget's live price preview
 * (BookingCard.tsx) and the checkout page's receipt (CheckoutPage.tsx).
 */
```

```
export function PriceBreakdownList({ rows }: PriceBreakdownListProps) {
  return (
    <Fragment>
      {rows.map(({ label, value, textClass = "text-muted" }) => (
        <li
          className={ list-group-item d-flex justify-content-between
                      align-items-center px-0 py-2 border-0 bg-transparent
        ${textClass}`}
        >
          <span>{label}</span>
          <span>{value}</span>
        </li>
      ))}
    </Fragment>
  );
}
```

Bemerkenswert ist, dass dieselbe Komponente von zwei völlig unterschiedlichen Aufrufstellen wiederverwendet wird (`BookingCard.tsx` und `CheckoutPage.tsx`), ohne dass sie etwas über ihren Aufrufkontext wissen muss — ein direkter Beleg dafür, dass sich echte Wiederverwendbarkeit erst einstellt, sobald eine Komponente rein aus Eingabedaten ein Ergebnis berechnet, statt selbst Zustand im DOM zu suchen und zu mutieren.

6.3 Dokumentationsstandard: JSDoc am Beispiel der JSX-Runtime

Als Referenz für den im Projekt angestrebten JSDoc-Standard dient die zentrale `createElement`-Funktion selbst — das Herzstück der eigenen JSX-Runtime (vollständige Datei: `frontend/src/utils/createElement.ts`):

```
/**
 * The JSX factory function (configured as the JSX pragma for this project).
 * For every JSX tag the compiler sees, it generates a call to this function
 * instead of `React.createElement`.
 *
 * Behavior depends on `tag`:
 * - If `tag` is a function (i.e. a component like `SkeletonCard`), it's
 *   simply called with the given attrs/children and its result is returned.
 * - If `tag` is {@link Fragment}, a `DocumentFragment` is created and all
 *   children are appended to it directly (no wrapper element).
 * - Otherwise `tag` is a plain HTML/SVG tag name (e.g. `"div"`), and a real
 *   element is created, attributes are applied to it, and children are
 *   appended.
 *
 * @param tag HTML tag name, {@link Fragment}, or a component function.
 * @param attrs Attributes/props for the element, or `null`.
 * @param children Child nodes/values (elements, strings, numbers, arrays,
 *   etc.).
 * @returns The created DOM element, or a `DocumentFragment` for fragments/
 *   components that return one.
```

```

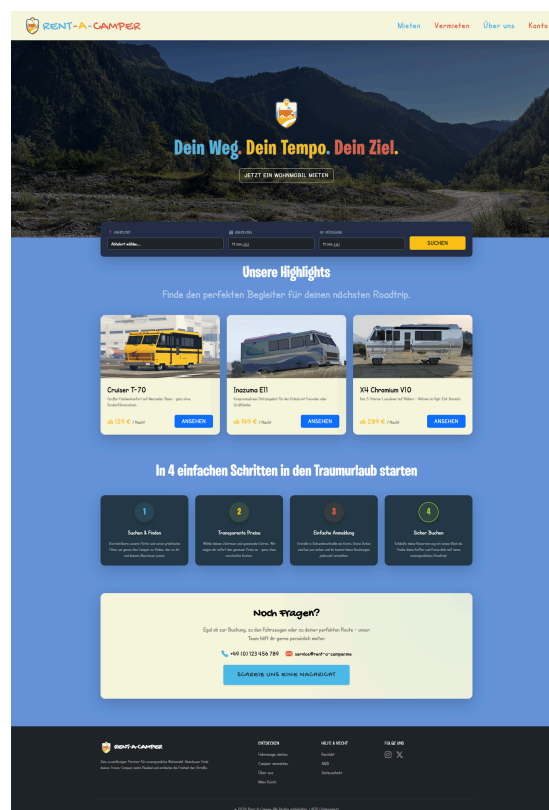
*/
export function createElement(
  tag: string | typeof Fragment | ((attrs: Attrs, ...children: unknown[]) =>
    Element | DocumentFragment),
  attrs: Attrs,
  ...children: unknown[]
): Element | DocumentFragment {
  // ... Implementierung siehe Kapitel 4.3 ...
}

```

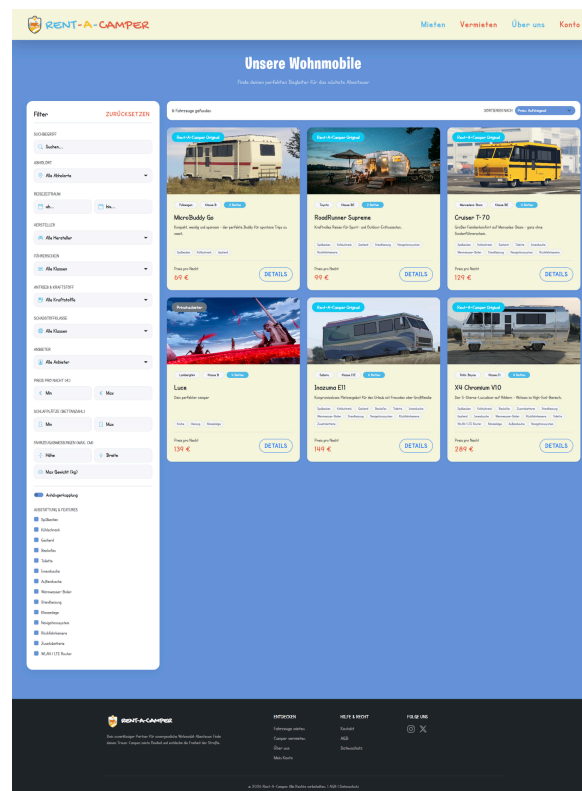
Dieser Dokumentationsstil — jede exportierte Funktion mit vollständigem `@param/@returns`, Querverweisen über `{@link ...}` und einer Erklärung des **Warum**, nicht nur des **Was** — wurde als verbindlicher Standard für alle im Zuge des Refactorings (Kapitel 5.3) neu geschriebenen oder überarbeiteten Dateien festgelegt und ist seither Teil des Code-Review-Kriterienkatalogs im Team.

7 Bildschirmabgriffe

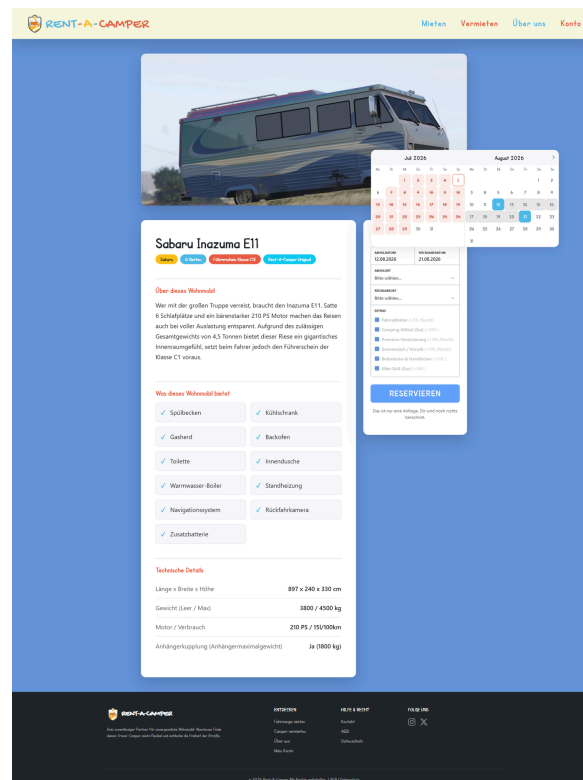
Die folgenden Screenshots stammen aus der tatsächlich laufenden Anwendung (lokaler Dev-Server, Desktop-Auflösung) und ordnen sich den in den Kapiteln 2 und 4 beschriebenen Techniken konkret aus Nutzersicht zu.



Startseite — Deklarative Komposition aus Hero-, Highlights- und Suchleisten-Komponente (Kapitel 4.4).



Miet-Übersicht mit Filter-Sidebar — RentPage.tsx mit FilterBar — Filter-/Sortierlogik über CamperFilterService (Kapitel 5.3, 6.2).



Camper-Detailseite mit Buchungs-Widget — Bildergalerie und BookingCard.tsx mit Flatpickr-Datumsbereich-Picker.

RENT-A-CAMPER

Mieten Vermieten Über uns Konto

Checkout

Fast geschafft! Überprüfe deine Daten und schicke die Buchung ab.

Deine Reiseplanung läuft ab in 14:48 Minuten

Persönliche Daten

VORNAME

NACHNAME

Dem

Kuhn

E-MAIL

john.doe@gmail.com

STANDORT/STADTLAND

Köln, DE

Zahlungsort

☒ Kreditkarte (Stripe)

Bitte bestätige mit Mastercard, Visa oder American Express

☐ PayPal

Bezahlst sofort mit deinem PayPal-Konto



Inzahlung E11

Abreise: 12.08.2024
Anreise: 20.08.2024

PREISZUSAMMENFASSUNG

148,00 € x 4 Nächte	592,00 €
Hauptkategorie / Aufschlag	+ 526,92 €
Kaufkraft (STV)	- 42,87 €
Platzierungsgebühr	150,00 €
Platzierungsversicherung	125,00 €
Servicekosten / Service	40,00 €
Gesamt	2158,53 €
Kaufkraft (STV an Nachzahlung)	1930,99 €

☒ Ich habe die **Reiseversicherung (REISE)** mit der **Reiseversicherung (REISE)** gelesen und akzeptiere diese.

ZAHLUNGSPFLICHTIG BUCHEN



RENT-A-CAMPER

Das zuverlässigste Partner für your private Wohnmobil. Rent-A-Camper ist die führende Marke für Wohnmobile in Deutschland.

REISEN

Fahrten planen

Camper mieten

Über uns

Wohnmobil

HEILIGEN

Kontakt

Alle

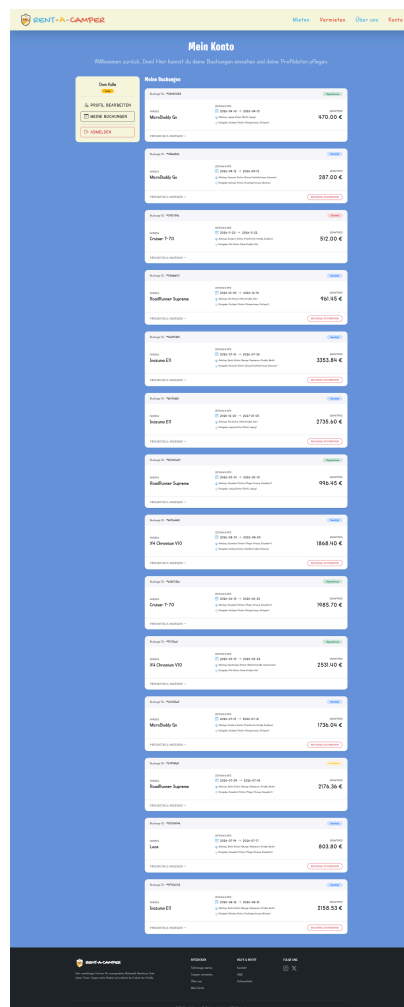
Dienstreise

FOLGE UNS

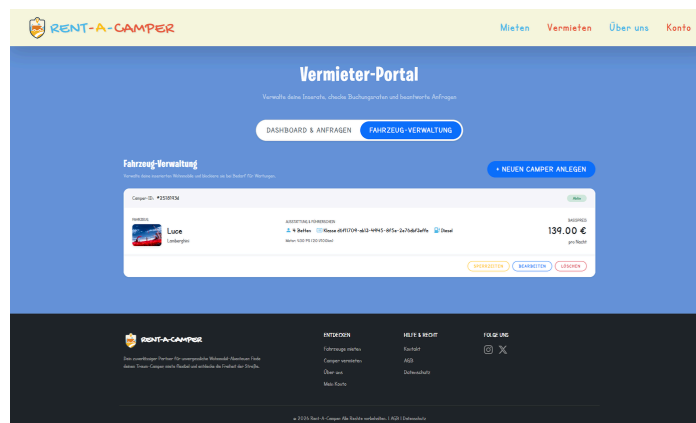


© 2024 Rent-A-Camper AG. Alle Rechte vorbehalten. | 100% Unternehmen

Checkout-Seite — CheckoutPage.tsx: Personendaten, Zahlungsmethode (PayPal/Stripe) und PriceBreakdownList (Kapitel 6.2).

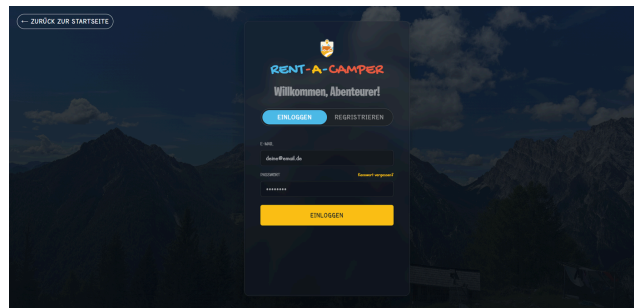


Konto-Bereich — Meine Buchungen — AccountPage.tsx/BookingsTable.tsx mit Status-Kennzeichnung je Buchung.



Vermieter-Dashboard — ProviderDashboard.tsx mit der Liste der eigenen Fahrzeuge (CamperListItem.tsx).

Formular zum Anlegen/Bearbeiten eines Wohnmobils — `CamperFormModal.tsx` aus `CamperCRUD.tsx` — Eingabefelder inkl. Bild-Upload.



Login / Registrierung — `AuthPage.tsx` — Authentifizierung gegen Supabase Auth (Kapitel 2.3).

8 Fazit und Ausblick

8.1 Fazit

„Rent-A-Camper“ demonstriert, dass sich auch ohne ein etabliertes Frontend-Framework eine komponentenbasierte, deklarative UI-Architektur umsetzen lässt — vorausgesetzt, die selbstgebaute Abstraktion (hier: die eigene JSX-Runtime) wird konsequent als solche respektiert und nicht durch imperative Direktzugriffe unterlaufen. Der in Kapitel 5 dokumentierte Irrweg war dabei kein Ausrutscher am Rande, sondern der Lernmoment des gesamten Projekts: die containerbasierte Zwischenlösung hat rückblickend mehr Entwicklungszeit gekostet (Bughunting am 01.07. plus das große Refactoring am 02.07., zusammen über 13 Stunden) als eine von Anfang an konsequent deklarative Umsetzung gebraucht hätte. Gleichzeitig zeigt das Backend mit seiner Schichtenarchitektur (Controller/Service/Domain/ Repository) und dem konsequenten Einsatz von Dependency-Injection-Tokens, dass sich die im Studium vermittelten Entwurfsprinzipien direkt auf ein NestJS-Projekt dieser Größenordnung übertragen lassen.

8.2 Ausblick

Aus der ehrlichen Bestandsaufnahme in Kapitel 3.1 und 5.3 ergeben sich folgende konkrete nächste Schritte:

- **Rest-Bereinigung von `RentPage.tsx`.** Die verbliebenen `container.querySelector`-Zugriffe sollten nach demselben Muster wie `PriceBreakDownList` in kleinere, rein deklarative Unterkomponenten überführt werden.

- **Konsolidierung der API-Schicht.** Die parallel existierenden Ordner `src/api/` (funktional) und `src/infrastructure/api/` (klassenbasiert) sollten zu einer einzigen Schicht zusammengeführt werden.
- **Den `shared/`-Ordner entweder reaktivieren oder entfernen.** Aktuell ist er totes Gewicht (Kapitel 3.1): entweder wird der `@shared/*`-Alias endlich produktiv genutzt (z. B. für tatsächlich geteilte DTOs), oder der Ordner wird ersatzlos gestrichen — inklusive der zugehörigen Anpassungen in `tsconfig.json` und `Dockerfile.backend`.
- **`class-validator` an der Backend-Grenze nachrüsten,** um die in Kapitel 6.1 offen benannte Lücke bei der DTO-Validierung zu schließen.
- **Erweiterungen für den Produktivbetrieb:** eine Merkliste über `localStorage`, eine Standort-Karte auf der Detailseite, sowie ein vollständiger Freigabe-Workflow für neu angelegte Fahrzeuge (Vermieter → Admin-Freigabe) — das `role`-Feld im Auth-Modell existiert dafür bereits als Datenstruktur.

Diese Punkte sind bewusst nicht als Mängel, sondern als nächste, klar umrissene Arbeitspakete zu verstehen — eine direkte Fortsetzung der in diesem Dokument beschriebenen Arbeitsweise: erst ehrlich den Ist-Zustand benennen, dann gezielt refactorn.